ASTROFY

Guia do usuário

Do pedido da página à implementação validada

Setup, entrevista, especificação, plano técnico,
execução e checkup em projetos Astro.

Edição v0.10.0
Gerado em 12 de outubro de 2026
Fonte editorial: docs/user/

Sumário

Astrofy: guia do usuário 4
- Comece aqui 4
- Consulte pela tarefa 4
- Formatos 4

Visão geral 5
- Artefatos do projeto 5

Instalação e setup 7
- Requisitos 7
- Instale o pacote e as skills 7
- Diga o que quer fazer 7
- Resultado da instalação 8

Planejamento de uma página 9
- Pontos de revisão 9

Entrevista e conteúdo 10
- Informações comuns 10
- Como responder 10
- Conduza a entrevista na conversa 10
- Alternativa pelo CLI 10
- Exemplo curto 11

Plano de implementação 12
- Aprovação 12

Execução e validação 13
- Conferência mínima 13

Exemplos por tipo de página 15
- Landing page de vendas 15
- Página de produto 15
- Página de serviço 15
- Página inicial 15
- Página Sobre 16
- Página de contato 16
- Página de preços 16
- Resultado comum 17

Execução retomável ... 18
Estados da tarefa ... 18
Passagem prática ... 18
Retomada em outra sessão ... 19
Tarefas do site com sitespec ... 20
Criar uma tarefa ... 20
Checklist e identidade do item ... 20
Acompanhar o progresso ... 20
Executar a tarefa ... 21
Dependências entre tarefas ... 21
Retomada e manutenção ... 22
Atualização do Astrofy ... 22
Solução de problemas ... 23
O projeto não é reconhecido ... 23
O setup encontrou JSON inválido ... 23
O tipo de página ficou ambíguo ... 23
A implementação divergiu do conteúdo aprovado ... 23
O plano ficou desatualizado ... 23
O Chromium não está disponível ... 23
A migração pendente deve falhar no CI ... 23
O ebook não confere ... 24

Astrofy: guia do usuário

Use o Astrofy em uma conversa com Codex ou Claude. Depois de instalar o pacote e disponibilizar as skills no projeto, descreva o trabalho com suas palavras. O agente examina o site, faz as perguntas necessárias e conduz o trabalho pelas skills adequadas.

Comece aqui

1. Instale o pacote e as skills.
2. Veja como a conversa conduz o trabalho.
3. Descreva ao agente a página ou a mudança desejada.

Os demais capítulos explicam cada etapa quando você quiser acompanhar o que o agente está fazendo ou operar o CLI manualmente.

Consulte pela tarefa

- **Definir uma página:** planejamento e entrevista de conteúdo.
- **Revisar a implementação:** plano técnico e execução e validação.
- **Consultar exemplos:** sete tipos de página.
- **Continuar depois:** execução retomável e retomada e manutenção.
- **Registrar uma tarefa pontual do site:** sitespec.
- **Resolver uma falha:** solução de problemas.

O manual do CLI descreve os comandos opcionais para instalação de skills, estado, checks e entrevistas pelo terminal.

Formatos

Os capítulos deste diretório são a fonte canônica. O projeto gera a mesma edição em PDF e EPUB dentro de `ebook/`. A versão do manual acompanha a versão do pacote Astrofy.

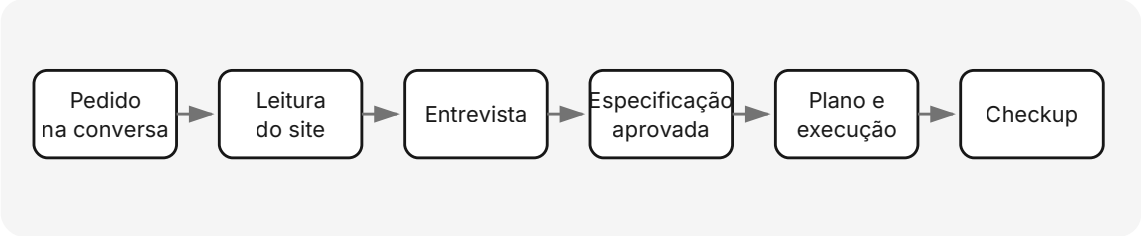
Visão geral

Você descreve o trabalho ao agente na conversa. O Astrofy examina o projeto, seleciona as skills necessárias e solicita as informações que faltam. Para criar uma página, a conversa aprova o conteúdo antes de passar ao plano técnico e à implementação.

O percurso de uma página tem três etapas:

1. `astrofy-page-planner` identifica o tipo de página e conduz a entrevista.
2. `astrofy-implementation-planner` relaciona o conteúdo aprovado à estrutura atual do projeto e apresenta um plano técnico.
3. As skills selecionadas implementam e conferem as tarefas aprovadas.

`astrofy-setup` organiza a adoção ampla ou a revisão inicial do site. Para uma página específica, explique o que quer fazer. O agente confere se precisa preparar o projeto antes de continuar.

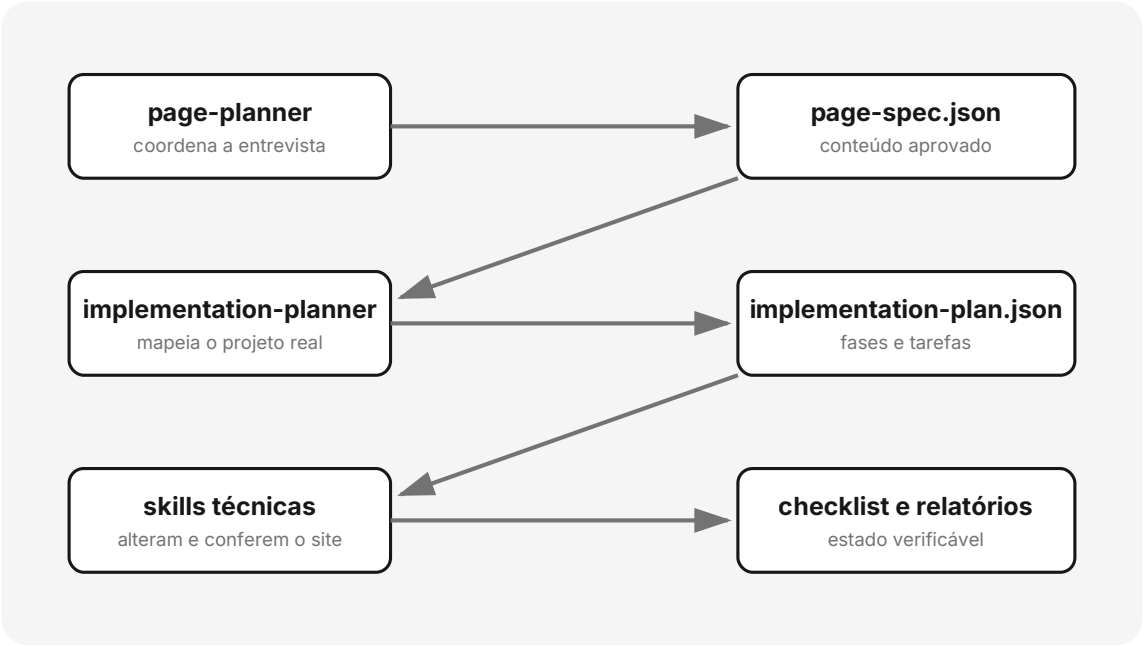


Etapas de uma página Astro

Você pode começar com uma frase, como “quero uma página para meu produto”. O agente confirma o tipo, coleta o conteúdo por etapas e apresenta alternativas quando precisa da sua escolha. Você revisa a página completa e o plano antes da implementação.

Artefatos do projeto

Caminho	Conteúdo
`.astrofy/config/project.json`	Configuração criada durante a adoção do Astrofy
`.astrofy/pages/<slug>/page-spec.json`	Especificação estruturada da página aprovada
`.astrofy/pages/<slug>/page.md`	Conteúdo completo para leitura e revisão
`astrofy.checklist.json`	Estado das verificações executadas no site
`.sitespec/<id>-<slug>/SITEPEC.md`	Tarefa pontual de desenvolvimento
`.sitespec/<id>-<slug>-<task>/SITEPEC.md`	Tarefa técnica da página (`route` , `content` , `check`), com `page` e `task`



Relação entre orquestradoras, artefatos e skills

Instalação e setup

Requisitos

- Node.js 22.12.0 ou posterior.
- Um projeto Astro existente com `package.json`.
- Permissão de escrita no diretório do projeto.

Instale o pacote e as skills

```
npm install --global @promovaweb/astrofy
cd /caminho/do/site
astrofy skills install --agent codex
```

Troque `codex` por `claude` se esse for o agente usado no projeto. O comando coloca as skills em `.agents/skills/` ou `.claude/skills/`. Para conferir os arquivos antes de gravar, acrescente `--dry-run`.

Diga o que quer fazer

Na conversa aberta para o projeto, diga o que quer fazer. Por exemplo:

> Quero criar uma página de produto. Use o Astrofy para entender o projeto, conversar comigo sobre o conteúdo e mostrar o plano antes de implementar.

Para uma adoção ampla do framework, peça ao agente que use `astrofy-setup`. Essa skill reconhece as convenções e integrações do site, inicializa os contratos ausentes e registra o estado para retomadas. Para uma página específica, explique o que quer fazer. O agente confere o que precisa preparar antes de chamar `astrofy-page-planner`.

O agente executa as operações do Astrofy necessárias ao trabalho. Você não precisa rodar `astrofy inspect`, `astrofy init`, `astrofy page create`, `astrofy page answer` ou `astrofy plan` por separado. Use esses comandos diretamente apenas quando quiser conduzir a mesma etapa pelo terminal. O manual do CLI descreve esses comandos.

Depois de instalado, `astrofy` sem nenhum comando abre o painel interativo (capítulo execução retomável explica a aba de itens; o painel também cobre tarefas de `.sitespec/`). Em script ou CI, sem terminal interativo, o mesmo comando devolve a consulta equivalente a `astrofy status`.

Resultado da instalação

Depois da instalação, as skills ficam disponíveis no projeto. A adoção cria `.astrofy/` e `astrofy.checklist.json` quando o trabalho solicitado precisar desses contratos. Confira os arquivos alterados antes de aprovar a implementação.

Planejamento de uma página

Descreva a página ao agente na conversa. `astrofy-page-planner` examina a estrutura atual do site e escolhe uma especialista. O agente prepara os contratos necessários quando o projeto ainda não usa Astrofy.

Pedido	Especialista
Landing page de vendas	`astrofy-plan-sales-page`
Página de produto	`astrofy-plan-product-page`
Página de serviço	`astrofy-plan-service-page`
Página inicial	`astrofy-plan-homepage`
Página Sobre	`astrofy-plan-about-page`
Página de contato	`astrofy-plan-contact-page`
Página de preços	`astrofy-plan-pricing-page`

Se a solicitação combinar dois formatos, a orquestradora define um tipo principal e registra as necessidades complementares. Uma página de produto com tabela de planos, por exemplo, continua sendo produto e recebe requisitos de preços.

Pontos de revisão

Primeiro, o agente confirma a finalidade e o público e valida a arquitetura. Depois, organiza conteúdo e mídia e define os comportamentos da página. Você recebe a página completa para revisar. Quando precisa da sua escolha, o agente apresenta três opções diferentes e recomenda uma delas.

A saída segue `packages/schemas/page-spec.schema.json`. A implementação ainda não começa: primeiro você revisa `page.md` e aprova a especificação. O agente registra as respostas e a página nos arquivos do projeto.

Entrevista e conteúdo

A especialista pergunta somente o necessário para o tipo escolhido. Ela aceita texto pronto, notas incompletas ou autorização para propor alternativas.

Informações comuns

- finalidade e ação principal;
- público e contexto de chegada;
- áreas da página e ordem de leitura;
- títulos, textos, rótulos e mensagens de estado;
- imagens, vídeos, ilustrações e suas fontes;
- links, formulários e integrações;
- amostra de texto aprovada, tom desejado e expressões a preservar ou evitar;
- fatos que precisam de fonte;
- conteúdo ainda pendente.

Como responder

Você pode responder uma etapa por vez. Use `aprovar`, `ajustar` ou `manter pendente` para cada proposta. Quando fornecer um texto final, diga que ele deve ser preservado. Quando quiser ajuda editorial, informe os fatos disponíveis e o tom desejado. Você também pode fornecer uma amostra de texto aprovada ou descrever como quer que a página soe. O Astrofy registra essa orientação para as skills que vão implementar e revisar a copy.

Propostas escritas pelo agente permanecem como rascunho até sua aprovação. Copy fornecida e aprovada não será reescrita apenas para caber no layout.

Áreas opcionais podem ficar pendentes. A aprovação final registra seu estado e impede que uma lacuna seja preenchida silenciosamente durante a implementação.

Conduza a entrevista na conversa

Responda às perguntas do agente na ordem apresentada. Você pode aprovar, ajustar ou deixar uma área opcional pendente. O agente registra as respostas e monta a especificação para sua revisão.

Alternativa pelo CLI

Use estes comandos se quiser conduzir a mesma entrevista pelo terminal. Eles não são necessários quando o agente conduz a conversa.

Inicie a entrevista informando o tipo da página:

```
astrofy page create --slug atlas --type product
astrofy page status --slug atlas
```

A saída apresenta até três itens em `nextQuestions`. Registre cada resposta com o identificador recebido:

```
astrofy page answer --slug atlas --field title --value "Atlas"
astrofy page answer --slug atlas --field route --value /atlas/
```

O arquivo `interview.json` conserva respostas e lacunas. Quando todas as entradas obrigatórias existem, o CLI gera `page-spec.json` com conteúdo, SEO, integrações, acessibilidade e testes.

O schema atual do CLI não tem campos para amostras de voz. Registre a direção confirmada na seção `Voz e redação` de `page.md` durante a revisão da especificação.

Exemplo curto

```
Usuário: Quero uma página do produto Atlas.
Astrofy: A ação principal deve iniciar teste, solicitar demonstração ou comprar?
Usuário: Solicitar demonstração.
Astrofy: Vou propor três arquiteturas para uma página de produto orientada à demo.
```

Plano de implementação

Depois da aprovação da página, o agente chama `astrofy-implementation-planner`. A skill relê o projeto para relacionar cada área com arquivos existentes e mudanças necessárias.

O plano registra:

- rota e modo de renderização;
- direção de voz para tarefas que exibem texto;
- layouts, componentes e conteúdo reutilizados;
- arquivos que serão criados ou alterados;
- integrações e variáveis de ambiente;
- acessibilidade, SEO e dados estruturados;
- testes automáticos e revisões manuais;
- revisão editorial quando a tarefa cria ou altera copy;
- documentação afetada;
- dependências, ordem e estimativa de cada tarefa.

Cada tarefa é uma sitespec comum (contrato em `docs/sitespec.md`, schema em `packages/schemas/sitespec.schema.json`), com `page` e `task` no front matter apontando de volta para a página. Não existe mais um arquivo de plano separado: `astrofy sitespec list` / `progress` e a aba Specs da TUI leem essas tarefas junto de qualquer outra tarefa do site.

O agente gera o plano estruturado depois que a entrevista estiver pronta. Se quiser fazer essa etapa pelo terminal, use:

```
astrofy plan --slug atlas
```

O comando grava três tarefas mecânicas (`route`, `content`, `check`) em `.sitespec/`. `astrofy-implementation-planner` pode acrescentar outras, sempre com o mesmo `page` e um `task` próprio, quando a página exigir mais decomposição. As tarefas começam em `draft`; `blockedBy` (calculado a partir de `dependsOn`) mostra quais ainda esperam uma predecessora. Reexecutar `astrofy plan` reconcilia sem apagar progresso ou itens já marcados.

Aprovação

Confira se cada tarefa aponta para um resultado observável e uma forma de conferência. Ajuste o plano antes de executar quando o caminho proposto não combinar com a arquitetura atual. A aprovação libera as skills técnicas por fase.

Execução e validação

O plano indica as skills adequadas para cada tarefa. O agente executa as que se aplicam ao escopo aprovado. Uma sequência comum usa:

1. `astrofy-architecture`, `astrofy-routing` e `astrofy-config` para a base.
2. `astrofy-components`, `astrofy-sections` e `astrofy-page-design` para a interface.
3. `astrofy-images`, `astrofy-forms` ou `astrofy-react` conforme a página.
4. `astrofy-accessibility`, `astrofy-seo`, `astrofy-open-graph` e `astrofy-structured-data` para qualidade pública.
5. `astrofy-editorial-review` quando uma tarefa criar ou alterar texto visível. A skill pode auditar sem editar ou corrigir a copy dentro do escopo aprovado.
6. `astrofy-testing`, `astrofy-documentation` e `astrofy-checkup` para fechar o trabalho.

Cada skill lê o plano, confere a implementação atual, registra o que alterou e atualiza o estado da tarefa. Se preferir acompanhar as tarefas pelo terminal, use `apply` para marcar o início e a conclusão:

```
astrofy apply --slug atlas
astrofy apply --slug atlas --task route --status completed
```

Conferência mínima

```
npm run check
npm run build
astrofy check --root .
astrofy check --root . --browser
```

O primeiro `check` não abre navegador. A opção `--browser` inclui Chromium e confere temas, overflow, responsividade e erros de runtime no preview local.

O agente executa os checks aplicáveis e informa o que ainda precisa de revisão manual. Revise a página no navegador em larguras de celular e desktop. Formulários, menus, foco, links e mensagens de erro exigem interação real.

Registre o resultado da revisão manual com `astrofy tui` (tecla `m`) ou, sem terminal interativo, com `astrofy review`:

```
astrofy review --rule accessibility.alternatives --page /produtos/atlas/ --status
passed --reviewer "QA" --reason "Alt text confere com a imagem publicada."
```

Uma regra `automatic` não aceita esse registro; ela só é aprovada pelo próprio verificador em `astrofy check`.

Exemplos por tipo de página

Os exemplos abaixo mostram o pedido, as respostas centrais, a arquitetura aprovada e o foco do plano técnico. Nomes e textos são ilustrativos.

Landing page de vendas

Pedido: “Quero vender a formação Operação DevOps.”

Respostas centrais: profissionais de desenvolvimento; inscrição como ação principal; turma com data e vagas; depoimentos autorizados; pagamento externo.

Arquitetura: hero com oferta, transformação esperada, programa, formato, instrutor, depoimentos, preço, perguntas frequentes e chamada final.

Plano técnico: rota da campanha, componentes de prova e preço, integração do checkout, eventos da ação principal, metadados e testes do percurso.

Página de produto

Pedido: “Quero criar uma página para o Atlas, nosso produto de atendimento.”

Respostas centrais: equipes de suporte; demonstração como ação principal; capturas reais do produto; integrações confirmadas pela documentação.

Arquitetura: hero, problemas atendidos, casos de uso, recursos, demonstração visual, integrações, perguntas frequentes e chamada final.

Plano técnico: rota `/atlas/`, seções reutilizáveis, formulário de demonstração, imagens responsivas, `SoftwareApplication` em JSON-LD e testes.

Página de serviço

Pedido: “Preciso apresentar nossa consultoria de modernização.”

Respostas centrais: empresas com aplicações legadas; diagnóstico como ação; escopo por etapas; entregas e responsabilidades explícitas; contato comercial.

Arquitetura: cenário atendido, resultados, entregas, processo, requisitos do cliente, equipe, casos relacionados, perguntas e contato.

Plano técnico: composição com componentes existentes, formulário com qualificação, confirmação de envio, metadados `Service` e teste dos estados.

Página inicial

Pedido: “Quero reorganizar a Home para explicar a empresa.”

Respostas centrais: três públicos; produto como destino principal; serviços e conteúdos como rotas secundárias; mensagem institucional já aprovada.

Arquitetura: proposta central, caminhos por público, produto em destaque, serviços, recursos recentes, comprovação institucional e contato.

Plano técnico: revisão da rota inicial, navegação, seções compartilhadas, ordem responsiva, links internos, metadados da organização e regressão visual.

Página Sobre

Pedido: “Quero uma página Sobre que conte nossa trajetória.”

Respostas centrais: fatos documentados; marcos com datas; pessoas com autorização; princípios descritos por ações; contato para imprensa.

Arquitetura: atuação atual, história, marcos, pessoas, princípios, presença pública e canais institucionais.

Plano técnico: conteúdo estruturado, imagens com texto alternativo, schema `Organization`, links para fontes públicas e revisão de todas as afirmações.

Página de contato

Pedido: “Quero separar suporte de oportunidades comerciais.”

Respostas centrais: dois destinos; campos mínimos por assunto; consentimento; prazo de retorno informado; email como canal alternativo.

Arquitetura: escolha do assunto, formulário adaptado, orientações de privacidade, mensagens de envio, prazo de retorno e canais alternativos.

Plano técnico: validação no cliente e servidor, proteção contra abuso, integrações por ambiente, foco após envio, testes de falha e documentação.

Página de preços

Pedido: “Quero comparar três planos do produto.”

Respostas centrais: cobrança mensal e anual; limites por plano; impostos e moeda explícitos; teste gratuito; contato para volume maior.

Arquitetura: contexto da cobrança, seletor de período, cartões dos planos, tabela comparativa, condições comerciais, perguntas e ação por plano.

Plano técnico: fonte única para preços, seletor acessível, URLs do checkout, dados estruturados compatíveis com o conteúdo e testes das combinações.

Resultado comum

Cada exemplo termina com `page-spec.json`, `page.md` e as sitespecs da página (`route`, `content`, `check`) em `.sitespec/`. A implementação somente usa fatos e textos aprovados na entrevista.

Execução retomável

Cada tarefa da página é uma sitespec com `page` e `task` no front matter (contrato completo em `docs/sitespec.md`). O agente atualiza esse registro enquanto trabalha para que você retome em outra conversa. Este capítulo mostra os comandos do CLI para consultar e atualizar as tarefas pelo terminal.

Estados da tarefa

`astrofy apply --status` usa o vocabulário abaixo; internamente, cada um grava um `status` de sitespec diferente.

`--status` de `apply`	`status` da sitespec	Significado	Próxima ação permitida
(tarefa recém-criada)	`draft`	Ainda não começou	Conferir `blockedBy` e iniciar
`in_progress`	`in_progress`	Há trabalho em andamento	Continuar passos registrados
`completed`	`done`	Checklist inteira marcada	Libera dependentes
`failed`	`blocked`	Uma validação falhou	Corrigir e tentar de novo
`skipped`	`archived`	Saiu do escopo com justificativa	Manter a justificativa

Não existe mais um par `pending` / `ready` separado do `status` da tarefa: `astrofy sitespec list` / `progress` calculam `blockedBy` a partir de `dependsOn` a cada leitura — uma tarefa `draft` sem `blockedBy` pode começar; uma tarefa `draft` com `blockedBy` ainda depende de outra.

Passagem prática

`astrofy plan --slug atlas` grava a tarefa `route` como sitespec:

```
status: draft
dependsOn: []
relatedFiles:
  - src/pages/produtos/atlas.astro
```

Ao iniciar, a skill relê a rota, registra os arquivos observados e muda o status para `in_progress`:

```
astrofy apply --slug atlas --task route --status in_progress
```

Depois de editar, executa as validações da checklist e faz a revisão manual indicada. Só então registra a conclusão — isso marca `status: done` e cada item da checklist como `[x]`:

```
astrofy apply --slug atlas --task route --status completed
```

Se uma validação falhar, registre `failed` em vez de `completed`; a tarefa vai para `status: blocked` e a dependente continua com `blockedBy` não vazio. Uma nova execução volta à tarefa com falha antes de avançar:

```
astrofy apply --slug atlas --task route --status failed
astrofy apply --slug atlas --task route --status completed
```

Ao concluir uma tarefa, a dependente perde a entrada correspondente em `blockedBy` automaticamente — não é um campo gravado, é calculado a cada leitura. Sem `--task`, `astrofy apply --slug atlas` escolhe a tarefa em `in_progress`, ou a primeira `draft` sem `blockedBy`.

Retomada em outra sessão

1. Confira `git status` e os arquivos alterados.
2. Execute `astrofy page status --slug atlas` e leia `page-spec.json`.
3. Execute `astrofy sitespec list --json` (ou `sitespec progress`) e filtre as tarefas com `page: atlas` para localizar a próxima.
4. Localize tarefas `in_progress` ou `blocked`.
5. Confirme se os arquivos ainda correspondem ao registro.
6. Continue a tarefa e repita suas validações.
7. Atualize o estado com `astrofy apply` e confirme que a dependência saiu de `blockedBy` da próxima tarefa.

Tarefas do site com sitespec

Nem toda mudança no site é uma página nova. Um ajuste de componente, uma correção de contraste, uma pendência de SEO — isso vira uma tarefa em `.sitespec/`, não uma entrevista de página. Use `astrofy-page-planner` (capítulo 2) para páginas completas; use sitespec para o resto do trabalho de desenvolvimento do site.

Criar uma tarefa

Descreva o pedido ao agente. `astrofy-sitespec-create` confere se já existe uma tarefa igual, mapeia as skills relacionadas ao pedido e só pergunta o que sobrar sem resposta — a maioria dos pedidos não gera pergunta nenhuma. Quando o pedido é ambíguo em algo que mudaria a tarefa, a skill faz no máximo três perguntas objetivas, cada uma com uma opção recomendada, antes de escrever o arquivo.

```
Pedido: "o botão primário está com contraste baixo no tema claro"
→ sem pergunta: skill, arquivo e critério de pronto já estão no pedido
→ cria .sitespec/0003-corrigir-contraste-botao/SITEPEC.md
```

A tarefa criada fica em `status: draft`, com `primarySkill` (quem executa) e `reviewSkills` (quem contribuiu critério para a checklist).

Checklist e identidade do item

Cada item da checklist tem um id curto e fixo, `{#id}`, e pode marcar a skill responsável:

```
- [ ] Ajustar o token de cor do botão `#astrofy-design-system` {#a3f2}
- [x] Medir o contraste atual `#astrofy-accessibility` {#a001}
```

O id não muda depois de criado, mesmo que o item seja reordenado. Um item sem `{#id}`, ou com um id repetido na mesma checklist, invalida a tarefa inteira até a correção — o contrato completo está em `docs/sitespec.md`.

Acompanhar o progresso

```
astrofy sitespec list --json
```

Devolve as tarefas existentes com progresso (`done` / `total`), o próximo id livre e, para cada tarefa, `blockedBy` (dependências ainda não concluídas). Pelo terminal interativo, `astrofy tui` abre a aba **Specs** (`Ctrl+S`): lista as tarefas, mostra prioridade e progresso, e `Enter` abre o detalhe completo com descrição, checklist e notas. A aba é só consulta.

Executar a tarefa

`astrofy-sitespec-implement` lê a tarefa, muda `status` para `in_progress` , executa cada item com a skill marcada e marca `[x]` pelo `{#id}` ao terminar. Ao final:

Resultado	`status`
Toda a checklist concluída	`done`
Impedimento externo à tarefa	`blocked` , com `blockedReason`
Decisão de não terminar	`archived` , com `archivedReason`

Decisões e desvios ficam registrados em `## Notas` , datados.

Dependências entre tarefas

Uma tarefa pode declarar `dependsOn` com o id de outra que precisa terminar primeiro. `astrofy sitespec list --json` resolve isso automaticamente:

```
astrofy sitespec list --json
```

A tarefa dependente aparece com `blockedBy: ["0001"]` enquanto a 0001 não estiver `done` . Isso é informativo — não muda `status` para `blocked` sozinho; a pessoa ou a skill decide se a dependência pendente impede avançar mesmo assim.

Retomada e manutenção

Os arquivos em `.astrofy/` permitem continuar o trabalho em outra sessão. Abra uma conversa no projeto e peça ao agente para retomar a página pelo slug. Ele confere a especificação, o plano, o Git e continua pela próxima tarefa cujas dependências estejam concluídas. Use `astrofy page status` e `astrofy apply` quando quiser retomar pelo terminal.

Se o conteúdo mudar antes da implementação, volte à especialista da página, atualize a especificação e gere novamente as partes afetadas do plano. Se a mudança surgir durante a execução, registre seu impacto antes de editar código.

Atualização do Astrofy

```
npm update --save-dev @promovaweb/astrofy
npx astrofy --version
npx astrofy inspect --root .
```

Leia o `CHANGELOG.md` entre a versão instalada e a nova edição. Execute as validações do projeto depois da atualização.

Solução de problemas

O projeto não é reconhecido

Execute `astrofy inspect --root <diretório>` apontando para a pasta que contém o `package.json` do site. Confira se `astro` aparece em `dependencies` ou `devDependencies`.

O setup encontrou JSON inválido

Abra o caminho informado, corrija a sintaxe e repita `astrofy init --dry-run`. O comando preserva arquivos existentes e interrompe a gravação quando não pode interpretá-los.

O tipo de página ficou ambíguo

Informe a finalidade principal e a ação esperada. A orquestradora usa essas duas respostas para escolher a especialista e mantém necessidades secundárias na especificação.

A implementação divergiu do conteúdo aprovado

Pare a tarefa, compare a página com `brief.md` e `page-spec.json` e corrija a implementação. Mudanças editoriais precisam voltar à etapa de conteúdo.

O plano ficou desatualizado

Execute `astrofy plan --slug <slug>` após reler o projeto. O comando recria as tarefas a partir do contrato atual da página. Preserve no Git o plano anterior quando precisar consultar o histórico das tarefas concluídas.

O Chromium não está disponível

Consulte a instalação controlada pelo Astrofy:

```
astrofy browser status
astrofy browser install
```

Depois execute `astrofy check --browser`. No CI, mantenha o cache indicado no workflow distribuído para evitar baixar o Chromium a cada execução.

A migração pendente deve falhar no CI

Use o modo de conferência:

```
astrofy migrate --check
```

O comando retorna código 1 quando o contrato precisa de migração e código 0 quando a configuração já usa a versão atual.

O ebook não confere

Na raiz do Astrofy, execute:

```
npm run ebook
npm run ebook:verify
```

O primeiro comando recompila PDF, EPUB e manifesto. O segundo compara a edição com todos os capítulos e arquivos do pipeline.